Reconstruction Order, Not Solver Family, Governs Control Accuracy: A Comparison of Neural and Classical HJB Solvers Against an Exact Reference

Jingwen Zhang
School of Mathematics and Physics
Xi'an Jiaotong-Liverpool University
Suzhou, China

Fei Ma
School of Mathematics and Physics
Xi'an Jiaotong-Liverpool University
Suzhou, China

***Abstract*—Value accuracy does not bound control accuracy. On the Hamilton–Jacobi–Bellman (HJB) equation of optimal execution, the reconstruction that turns nodal values into a gradient can dominate the error in the strategy actually executed. With a multilinear reconstruction the control error is 10 to 684 times the value error. Replacing it with a cubic reconstruction on the same nodes is four to five orders of magnitude more accurate and costs nothing at the grid level. What governs control accuracy is therefore the order of the reconstruction, not the choice of solver family.**

This reframes the choice between neural and classical solvers, which is what we set out to measure. Physics-informed neural networks (PINNs) are widely proposed for optimal control in finance on the grounds of the curse of dimensionality, but it is unclear when a neural solver is the right choice and when a classical one is. We compare explicit finite differences, Howard policy iteration and two PINN variants on a multi-asset optimal-execution HJB whose value function has an exact matrix Riccati reference, both without a predictive signal and with one. Because that reference is exact rather than another solver, it removes the circularity of scoring one solver against another, and it stays exact at every state dimension — so accuracy can be measured where no classical baseline can run.

Against that reference the comparison gives a boundary rather than a winner. Where a grid scheme can run ($d \leq 5$) it is both faster and more accurate, by a factor of 5 to 50 in time at $d = 4$–5. Beyond that the mesh-free solver is the only method that runs, at accuracy decaying from $R^2 = 0.98$ at $d = 8$ to 0.76 at $d = 20$ and with no internal signal of the decay. The neural solver's failure at high dimension is a property of *parameterisation*, not of dimension: the same architecture fails on a symmetric problem and succeeds on a non-separable one at matched state dimension.

***Index Terms*—Hamilton–Jacobi–Bellman equation, physics-informed neural networks, neural operators, optimal execution, numerical methods, solver benchmarking, method selection.**

I. Introduction

Deep learning methods for partial differential equations (PDEs) are widely proposed for optimal control in finance on the grounds that mesh-based solvers suffer from the curse of dimensionality. In high-frequency trading (HFT) the relevant problem is the Hamilton–Jacobi–Bellman (HJB) equation of optimal execution, whose state is the trader's inventory and, when a predictive signal is modelled, the signal that drives it. The natural question is practical rather than rhetorical: should such a problem be solved with a neural solver or a classical one, and does the answer change with dimension? This paper answers by running both against an answer that is known exactly, both without a signal and with one.

The comparison is worth making carefully. Much of the applied literature scores a solver against *another solver*, which makes the reference exact by construction and its reported error zero; a closed-form solution removes that circularity. Most comparisons also report one accuracy number, the value function, when the quantity an HJB is solved *for* is the control — a derivative, obtained differently by each family and, as it turns out, not equally accurately. And most are run in a single regime, whereas the answer here depends on the regime.

This paper compares two solver families on a benchmark whose answer is known in closed form, at matched accuracy, scored on *both* the value function and the control. Three findings follow. First, the comparison yields a boundary rather than a winner: a grid scheme is faster and more accurate wherever it is runnable ($p \leq 5$), and beyond that the mesh-free solver is the only method that runs, at accuracy that decays with dimension without any internal warning. Second, and more usefully, value accuracy does not bound control accuracy: the control error is governed by the order of the reconstruction the grid solver differentiates, not by the accuracy of the solution, so a cubic reconstruction on the same nodes buys four to five orders of magnitude for free. Third, the neural solver's failure is a property of the problem's parameterisation rather than its dimension. We close with a decision guide: which regime favours which method, and the measurement that decides it.

II. Related Work

Three literatures meet at this paper's question: how the optimal-execution problem is *formulated*, how it has been *solved*, and what the recent mesh-free turn has changed. The

formulation is settled; the solver choice is where the question lies open.

Formulation. Avellaneda and Stoikov derived closed-form approximations for market-making control under inventory risk [1]. Guéant *et al.* formulated optimal execution under market impact as a nonlinear HJB problem [2], and Guéant extended the closed-form approximation to several assets [3]. Cartea and Jaimungal added risk criteria and microstructure features [4], and Almgren and Chriss gave the canonical mean–variance trajectory [5], which is the forced-liquidation limit of our benchmark and is treated at book length in [6]. Everything this paper measures sits inside that formulation; the coefficients are synthetic, and we return to what that costs us in Section VI.

Solving it. The classical treatments are monotone finite-difference schemes [7] and policy iteration [8], both accurate and well understood but scaling poorly in the number of state variables. Their convergence theory is Kushner and Dupuis [9], and the dynamic-programming background for the value iteration we compare against is [10]. The poor scaling is the stated motive for the neural turn: PINNs embed the PDE residual in the loss [11], alongside the deep Galerkin and deep Ritz methods [12], [13] and deep BSDE solvers [14], [15]. Within that last family the variants differ in how the control is represented and in what is regressed: the deep BSDE scheme of [16], the deep splitting method [17], and the Malliavin-weighted formulations of [18], with [19] surveying the stochastic-control setting. We measure the plain deep BSDE scheme of [14], [15] in Table V, so the family is represented by its reference member rather than by its strongest variant – a distinction worth stating, since the variants were introduced precisely to address the training behaviour we report. High-order derivatives are the recognised bottleneck [20], and domain decomposition and adaptive refinement reduce training cost [21], [22].

The mesh-free turn, and the two gaps it leaves. The strongest precedents for *scale* are mesh-free and single-instance: the deep Galerkin method reached a 21-dimensional HJB, and a penalty formulation of the coupled HJB–continuity system solved mean-field games in 100 dimensions [23]. Those are arguments for mesh-free representation, which we do not dispute. The same representation choice is now standard for hedging and control directly rather than for the value function [24], a neighbouring problem with the same cost question and no classical baseline. Learned solution *operators* are the more recent line: FNO [25], [26], CNO [27], Transolver [28] and others map function spaces rather than solving one instance. Their cost claims are framed at *inference* time, and that framing is legitimate rather than a strawman: the classical baseline must be re-solved for every new instance. FNO states this directly, and UGrid concedes it from the other side [29]. That is why benchmark work reports training and inference cost separately [30], and why this paper reports both. No paper in that corpus applies an operator to HJB, optimal control or Bellman equations, so the gap addressed here is the absence of an accuracy-matched, exactly-referenced cost comparison at HFT-relevant dimensions.

Two methodological gaps follow, and they set this paper's design. First, most comparisons score a solver against *another solver*, which makes the reference exact by construction and its reported error zero. Removing that circularity needs an external exact solution, and the optimal-execution HJB has one in closed form. Second, most comparisons report a single accuracy number, the value function, when what an HJB is solved *for* is the control. The control is a derivative, each family obtains it differently, and it is not obtained equally accurately — which is the finding we lead with, and which no comparison design built on a single accuracy number can detect. It is worth being explicit that our comparison also excludes an entire classical alternative. Backward dynamic-programming methods that avoid a mesh by simulating paths and regressing the continuation value — the least-squares Monte Carlo family [31], treated at book length in [32] — reach high dimension without a grid and are the natural third family to place against the two we measure. We do not run them, and the reason is scope rather than judgement: our object is the comparison of a learned solver against a mesh-based one on a problem with an exact reference, and adding a regression-based classical method would widen the paper without changing what it measures. A reader deciding between deep BSDE and least-squares Monte Carlo should treat that comparison as open. Finally, the SHoP framework [33] has been applied to this HJB in earlier work. We do not claim SHoP, or any solver here, as a new method: the findings below concern the choice between existing families rather than the introduction of another.

III. Benchmark Problem and Analytic Reference

Let $x \in \mathbb{R}^d$ denote the state and $u \in \mathbb{R}^m$ the trading rate, with linear dynamics and additive noise,

$$dx = (Ax + Bu)\,dt + \Sigma\,dW. \tag{1}$$

The trader maximises expected risk-adjusted proceeds,

$$\sup_u \ \mathbb{E}\left[-\tfrac{1}{2}\int_0^T \big(x^\top Q x + u^\top R u\big)dt - \tfrac{1}{2}x_T^\top Q_T x_T\right], \tag{2}$$

where $u^\top R u$ is temporary market impact ($R = \eta I$), $x^\top Q x$ is an inventory risk penalty ($Q = \lambda\sigma^2 I$), and the terminal term penalises residual inventory. These coefficients are set synthetically rather than calibrated to an instrument: the object here is the behaviour of solvers on a problem with a known answer, and a calibrated η and an empirically estimated bid–ask spread would change the numbers without changing the comparison. The standard estimators for that calibration step, if it is wanted, are Roll's implicit spread [34] and the high–low estimator of Corwin and Schultz [35]. In the inventory-only case the state is the inventory itself, $x = q \in \mathbb{R}^p$ across p assets, so $d = m = p$; with $A = 0$, $B = I$ this is the Almgren–Chriss problem in LQ form and reproduces the

classical trajectory. The HJB equation is

$$-\partial_t V = \max_u \Big\{ -\tfrac{1}{2}\big(x^\top Q x + u^\top R u\big) + (\nabla V)^\top (Ax + Bu) \Big\} + \tfrac{1}{2}\mathrm{tr}\big(\Sigma\Sigma^\top \nabla^2 V\big), \tag{3}$$

with $V(x,T) = -\frac{1}{2}x^\top Q_T x$. Substituting the quadratic ansatz $V(x,t) = -\frac{1}{2}x^\top P(t)x + c(t)$ gives the matrix Riccati equation and an additive constant,

$$-\dot{P} = Q + PA + A^\top P - PBR^{-1}B^\top P, \qquad P(T) = Q_T, \tag{4}$$

$$\dot{c} = \tfrac{1}{2}\,\mathrm{tr}\big(\Sigma\Sigma^\top P\big), \qquad c(T) = 0, \tag{5}$$

and the optimal control

$$u^\star(x,t) = -R^{-1}B^\top P(t)\,x. \tag{6}$$

Eq. (6) is what makes the control testable without a sign convention to argue about: in the inventory-only case $P(t) \succeq 0$, so a liquidation problem with $q > 0$ has a *negative* optimal rate, and a solver that returns the positive of this quantity is inverted — any backtest built on it trades the wrong way in every period. In the scalar case P has the closed form $P(t) = \eta k\,[Q_T \cosh k(T{-}t) + \eta k \sinh k(T{-}t)]/[\eta k \cosh k(T{-}t) + Q_T \sinh k(T{-}t)]$ with $k = \sqrt{Q/\eta}$; as $Q_T \to \infty$ it tends to $\eta k \coth(k(T-t))$ and the optimal trajectory to the classical $q_t = q_0 \sinh(k(T-t))/\sinh(kT)$. We use both limits as independent checks on the reference.

A. A predictive signal

A trader with no view should trade only to manage inventory, so the benchmark above is economically natural but limited. Adding a mean-reverting signal, with $dq = u\,dt$ and $ds = -\kappa s\,dt + \sqrt{2\kappa}\,dW$, and paying the trader for holding inventory in the signal's direction keeps the problem linear–quadratic, so the reference stays exact: the state becomes (q, s), giving $d = 2p$ while $m = p$, and Q becomes *indefinite* rather than positive definite. Two consequences matter here. Separability is broken by construction rather than by randomising coefficients — with perfectly uniform parameters $\|P_{\text{off}}\|/\|P_{\text{diag}}\|$ reaches 0.205, against *exactly* zero in the uniform inventory-only case — and because P is indefinite the control has no fixed sign, so (6) is checked pointwise against the reference instead of against a sign rule. We use $\kappa = 1$ and a signal scale of 0.005, putting the target inventory for a unit signal near 0.5. No elementary closed form survives once $A \neq 0$, so the released code validates this reference by the same route as above.

The state dimension d is free, so the curse-of-dimensionality claim can be measured rather than asserted, and every method solves the same equation, so the comparison isolates representation and discretisation.

IV. Methodology

The four methods fall into two families on the one axis this paper is about. A grid solver stores V at N^d nodes and must *reconstruct* a continuous function from them before it can differentiate one; a network *is* a continuous function, so automatic differentiation returns its gradient exactly. Everything the paper reports about control accuracy follows from that difference, and from the order of the reconstruction the grid solver uses.

Solvers. The *finite-difference* scheme uses explicit backward time stepping on a tensor-product grid of N points per dimension, with the analytic Hamiltonian maximiser and central differences in space. *Policy iteration* uses Howard's algorithm on the same grid: policy evaluation solves the linear frozen-policy PDE and policy improvement sets $u = R^{-1}B^\top \nabla V$. Two implementation points matter. Because the frozen-policy term is genuine advection, the evaluation step must be upwinded — central differencing of a frozen advection term is unconditionally unstable. And the policy must be stored as a function of (x,t), because $P(t)$ is strongly time-dependent; collapsing it to a single array made the evaluation PDE inconsistent near $t = T$.

The PINN. The network $\mathcal{N}$ maps $(x,t) \mapsto V$ and is trained by minimising the HJB residual

$$r(x,t) = \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 \Delta V - \tfrac{1}{2}\big(x^\top Q x + u^{\star\top} R u^\star\big) + \big(\nabla V\big)^\top \big(Ax + Bu^\star\big), \tag{7}$$

with $u^\star = R^{-1}B^\top \nabla V$ substituted analytically, so the supremum over the control never has to be learned. Three design choices carry the accuracy. The terminal condition is imposed *exactly* rather than as a penalty, by writing

$$V(x,t) = V_T(x) + (T-t)\,\mathcal{N}(x,t), \tag{8}$$

so $V(x,T) = V_T(x)$ for any network parameters and no penalty weight has to be tuned against the residual; this matters more here than in a forward PDE, because the terminal region is where the control is largest. The residual is divided by its value at initialisation, so the loss starts at $O(1)$ and the same learning rate works at every p, and collocation points are drawn with density proportional to $(1-u)^\beta$ in time ($u \sim U(0,1)$, $\beta = 3$), concentrating samples near $t = T$ where $|\nabla V|$ is largest. Finally the inputs are scaled by an explicit component — the state box affinely to $[-1,1]$, time to $[0,1]$ — which is not cosmetic: it is the component whose omission produces the out-of-domain failure of Section VI-C. The architecture is a 4×64 $\tanh$ MLP trained with Adam [36]. *No architecture search is performed*: the network is fixed and is not tuned per dimension, so every neural statement here is a claim about a competently trained standard PINN rather than about the best available one. Section V-D then measures how much that choice matters.

Protocol and configuration. All metrics are computed against the analytic solution, never against another solver; value and control are reported separately because they turn out not to agree (Section V-B); and three reconstructions of the grid solution are compared — a piecewise-linear (multilinear) one probed by finite differences, that same reconstruction differentiated exactly, and a cubic one probed identically. The defaults ($\eta = 0.01$, $\lambda\sigma^2 = 0.01$, $Q_T = 0.1$, $\sigma = 0.01$, $T = 1$) give $k = 1$ and keep the control magnitude $O(1)$–$O(10)$, so the explicit time step is set by a realistic Courant–Friedrichs–

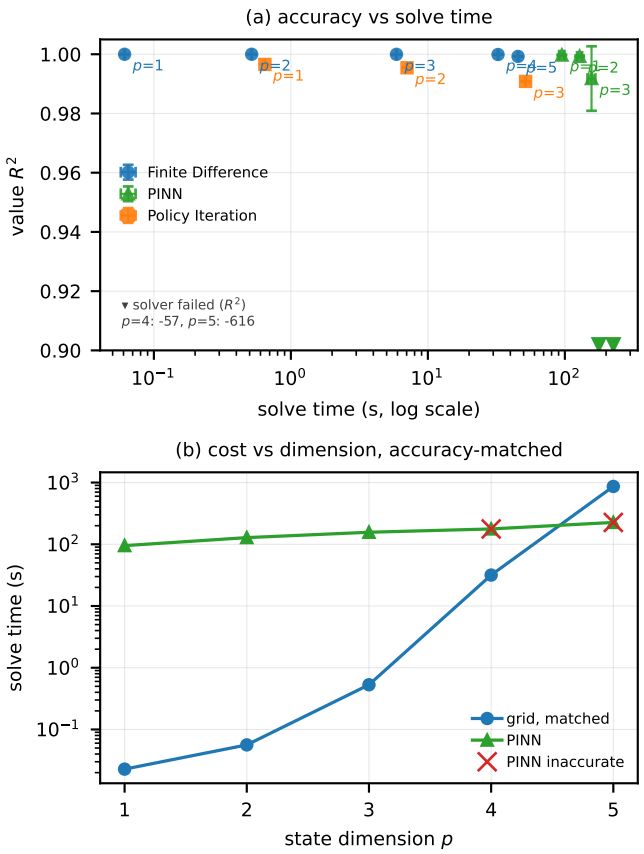


Fig. 1. Accuracy against solve time, both families on the machine each runs well on, with the accuracy-matched grid configuration ($N^\star = 21$) marked. Off-scale PINN failures are clipped and annotated rather than dropped, so a reader can see where the learned solver stops solving the problem instead of inferring it from an absence.

Lewy (CFL) bound rather than by an artificial stiffness. Neural training and timings run on a single NVIDIA RTX 4090 Laptop GPU (17.2 GB) and grid timings on the CPU. That asymmetry would make a wall-clock comparison unfair on its face, so the cost results are reported twice: as measured seconds, and as an analytic count of floating-point work with the rate each family achieves on the hardware it ran on (§V).

V. Results

Below, the dimension and cost results use the inventory-only benchmark, where $d = p$; the signal extension of Section III-A is used wherever a result could be an artefact of that model.

A. Accuracy and cost against the analytic solution

Table I is the central result. The finite-difference scheme is the most accurate method at every dimension, with value R^2 between 0.99927 and 0.99999, and its control has the correct sign at every sampled state. Policy iteration converges in 13 outer iterations and is slightly less accurate ($R^2 = 0.9908$ to 0.9966), as expected from the first-order upwind discretisation used in its policy-evaluation step.

The cost comparison does not depend on the hardware split. Both families were timed on the machine they run well on — the network on the GPU, the grid sweep on the CPU — so we also counted work rather than seconds. Counting $3d^2 + 3m^2 + 2dm + 4d + 2$ floating-point operations per grid point per time step, and $(6+2d)P$ per sample for the network ($P \approx 1.3 \times 10^4$ parameters for training and $2P$ per query), the grid scheme converts work into time at 0.13 to 0.27 GFLOP/s while the network reaches 26 to 30 GFLOP/s: a ratio of about 10^2. Counting work instead of seconds therefore *flatters the neural method*, which means the wall-clock result below is the conservative one and is not an artefact of giving the network a GPU. One caveat runs the other way and should be stated: the grid figures come from a NumPy implementation whose per-step cost is many small array operations, and its achieved rate is still rising with N rather than saturating, so a compiled kernel would be faster and would strengthen this conclusion. The absolute grid seconds should not be read as the best a grid scheme can do.

We had to reduce the grid resolution as p grew, so comparing cost at the resolutions the sweep could afford would be unfair. A separate refinement study swept N at each p and found the smallest resolution reaching a fixed accuracy target of $R^2 \geq 0.9999$. That resolution is $N^\star = 21$ at *every* dimension tested, which gives one accuracy-matched grid configuration per p; the grid columns of Table I use it. Grid work grows far faster than the point count N^d suggests, because the explicit time step is CFL-limited by the mesh spacing and work scales closer to N^{p+1}: at $p = 5$, $N = 21$ is 4.1×10^6 points over 420 steps, which is why accuracy-matched finite differences cost nearly 15 minutes at a dimension the neural solver covered in under four.

At matched accuracy the grid scheme is faster than the fixed-budget neural solver at every dimension: by a factor of 5.6 at $p = 4$ and by two to three orders of magnitude at $p \leq 2$. The mesh-free solver's cost is indeed almost flat in p (95 → 226 s, a factor of 2.4 across five dimensions) against 0.023 → 865 s for the grid scheme, so the two curves do cross — but at the crossing the neural solver is no longer solving the problem. Its accuracy is competitive at low dimension ($R^2 = 0.9997 \pm 0.0002$ at $p = 1$) and degrades with p, failing outright at $p \geq 4$ under this budget. A negative R^2 means the learned value function is worse than predicting the mean, so the method has not solved the problem rather than merely lost accuracy. Control-sign agreement stays high even where R^2 is negative: the network recovers the *direction* of the optimal control while missing its level, so sign agreement alone is not evidence that an HJB solver works. *That failure is specific to this parameterisation, not to the dimension* — see Section V-C.

B. Value accuracy is not control accuracy

An HJB is solved so that it can be *acted on*, and what is acted on is $u^\star = -R^{-1}B^\top \nabla V$ — a *derivative* of the value function. A grid scheme obtains it in two steps: reconstruct the stored nodal values as a continuous interpolant, then differentiate it. A network obtains it in one, by automatic differentiation, which is exact for whatever function the network represents. We call the first step the *reconstruction*, and its

TABLE I

ACCURACY AND COST BY STATE DIMENSION, AGAINST THE CLOSED-FORM RICCATI SOLUTION. THE GRID COLUMNS USE $N^\star$, THE SMALLEST RESOLUTION REACHING VALUE $R^2 \geq 0.9999$ AT THAT DIMENSION (FOUND BY REFINEMENT), SO THE TWO FAMILIES ARE COMPARED AT EQUAL ACCURACY RATHER THAN EQUAL RESOLUTION. PINN COLUMNS ARE A *fixed* BUDGET OF 6000 EPOCHS AND 2048 COLLOCATION POINTS; A LARGER BUDGET RESCUES $p = 4$ AND IS DISCUSSED IN THE TEXT. TIMES ARE GPU FOR THE PINN AND CPU FOR THE GRID SCHEME. A NEGATIVE R^2 MEANS THE LEARNED VALUE FUNCTION IS WORSE THAN PREDICTING THE MEAN, I.E. THE SOLVER HAS FAILED RATHER THAN MERELY LOST ACCURACY.

	grid scheme (accuracy-matched)				PINN (fixed budget)			
p	$N^\star$	time (s)	value R^2	ctrl R^2	time (s)	value R^2	ctrl R^2	sign agr.
1	21	0.023	0.99995	0.9958	95.1 ± 4.3	0.9997 ± 0.0002	0.9995 ± 0.0001	0.989 ± 0.007
2	21	0.056	0.99993	0.9957	128.2 ± 5.4	0.9993 ± 0.0003	0.9989 ± 0.0005	0.985 ± 0.008
3	21	0.530	0.99993	0.9958	156.6 ± 10.3	0.9918 ± 0.0109	0.9955 ± 0.0026	0.985 ± 0.005
4	21	31.710	0.99992	0.9959	177.1 ± 1.0*	-57.0218 ± 46.5787	-0.1707 ± 0.7816	0.996 ± 0.001
5	21	864.873	0.99991	0.9959	225.9 ± 4.4*	-615.7841 ± 0.6823	-1.6490 ± 0.0000	1.000 ± 0.000

*PINN accuracy has collapsed ($R^2 < 0.99$).

order the degree of the local polynomial piece it uses: one for a piecewise-linear (multilinear) reconstruction, three for a cubic one. A reconstruction of order k yields a gradient converging at a rate set by k, and it is that rate — not the accuracy of the solved value function — that governs the control error.

The two errors do not track each other. At every dimension and resolution tested the control error exceeds the value error, and it converges at a stable order of approximately 2.00: least-squares fits of $\log(\text{error})$ against $\log h$ give 2.00, 1.96, 1.96, 1.96 and 1.96 at $p = 1, \ldots, 5$, with fit R^2 between 0.998 and 1.000. The value error converges faster on average but irregularly, and at the finest grid in two of the five dimensions it regresses, so the ratio between the two grows under refinement — at $p = 2$ it runs $2 \to 5 \to 13 \to 25 \to 49 \to 83 \to 179$ over $N = 11, \ldots, 61$. The order is unchanged by the signal extension of Section III-A: 2.12 (fit $R^2 = 0.999$, over $N = 11, \ldots, 41$) against 2.10 for the inventory-only benchmark at the same state dimension, so what follows is a property of the differentiation operator and not of the inventory-only model.

The cause is the reconstruction, not the solution. We put the closed-form value function on the grid as exact nodal values, so the discrete solution is error-free by construction and every remaining control error belongs to the step that turns nodal values into a gradient. With the multilinear interpolation and probing difference used above, that step alone reproduces almost all of it: 3.7×10^{-3} against 3.8×10^{-3} at $N = 11$, falling to 1.1×10^{-4} at $N = 61$ at order 1.98, against $3.8 \times 10^{-3} \to 1.1 \times 10^{-4}$ measured on the solved grid. The grid solution is not what limits control accuracy.

What limits it is the order of the reconstruction. Table II measures the gradient-error convergence rate on functions whose gradients are known exactly, so the rates are properties of the reconstruction rather than of this benchmark. The multilinear reconstruction is second order on every function tested, and taking instead the *exact* gradient of that same interpolant — the best a piecewise-linear reconstruction can do — is also second order, so consistency with the interpolant buys nothing. A cubic reconstruction, being C^2, has a smooth rather than piecewise-constant gradient and is fourth order

TABLE II

THE ORDER OF THE RECONSTRUCTION GOVERNS CONTROL ACCURACY, NOT THE ACCURACY OF THE SOLUTION. EACH ROW TURNS NODAL VALUES INTO A GRADIENT A DIFFERENT WAY: A PIECEWISE-LINEAR (ORDER-ONE) RECONSTRUCTION WHOSE GRADIENT IS PROBED BY FINITE DIFFERENCES, THAT SAME RECONSTRUCTION DIFFERENTIATED EXACTLY, AND A CUBIC (ORDER-THREE) RECONSTRUCTION PROBED EXACTLY AS THE FIRST. THE COLUMNS REPORT THE CONVERGENCE RATE OF THE *gradient* — THE CONTROL — MEASURED ON FUNCTIONS WHOSE GRADIENTS ARE KNOWN EXACTLY, SO THE RATES ARE PROPERTIES OF THE RECONSTRUCTION RATHER THAN OF THE BENCHMARK; THE FIRST FOUR COLUMNS ARE THIS PAPER'S OWN PROBLEM, AVERAGED OVER $p = 2, 3$ AND 5. ORDERS AGREED ACROSS DIMENSIONS TO WITHIN 2.0. THE LAST COLUMN IS THE MERTON VALUE FUNCTION, A POWER FUNCTION FROM A GENUINELY NON-LINEAR CONTROL PROBLEM — NON-POLYNOMIAL, SO THE CUBIC RECONSTRUCTION IS NOT EXACT THERE. DIFFERENTIATING THE PIECEWISE-LINEAR RECONSTRUCTION EXACTLY IS NO BETTER THAN PROBING IT, SO CONSISTENCY WITH THE INTERPOLANT BUYS NOTHING. THE QUADRATIC COLUMN IS THIS PAPER'S OWN CASE AND FLATTERS THE CUBIC METHOD, WHICH REPRODUCES QUADRATICS EXACTLY; EVERY OTHER COLUMN IS THE ONE TO QUOTE. ERRORS ARE NORMALISED BY THE SPREAD OF THE TRUE GRADIENT.

	gradient-error convergence rate				
reconstruction	quad.	quartic	exp.	trig.	Merton
multilinear + probe	2.2	2.3	2.2	2.2	2.1
multilinear, slope	2.0	2.0	2.0	2.0	1.9
cubic + probe	exact	4.8	4.4	5.6	5.2

on the non-polynomial functions and four to five orders of magnitude more accurate on the same grid. Because this benchmark's value function *is* quadratic and a cubic reconstruction reproduces quadratics exactly, the last column applies the same comparison to the Merton problem, whose closed form is a power function and whose HJB is non-linear in the derivatives; there the cubic order is 5.2 against 2.1. The result is therefore not an artefact of the linear–quadratic structure.

This is why value and control accuracy can be chosen almost independently. On the benchmark's own solved grid at $p = 2$, $N = 61$, the multilinear extraction gives a control error of 1.1×10^{-4} against a value error of 6.1×10^{-7}, a ratio of 184. The cubic extraction gives 1.7×10^{-6} on the same grid, a factor of 67 better. There the control error is set by the solver rather than the reconstruction, so the ratio falls to 2.8. The decoupling is largely an artefact of the gradient operator chosen, not a property of grid methods.

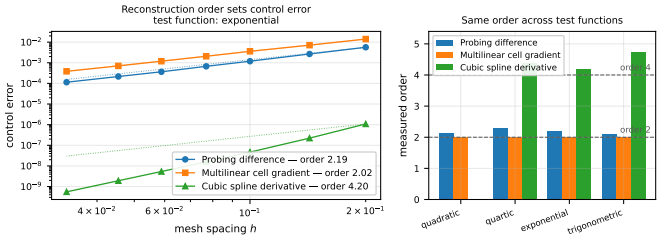


Fig. 2. Control error against mesh spacing. Left, the three reconstructions on one non-polynomial test function, with a slope-2 reference (dotted); the measured orders are fitted to the same data the curves show. Right, the same measurement across four test functions, so the order can be seen to be a property of the reconstruction rather than of the example. The cubic reconstruction is fourth order throughout; the quadratic case is omitted for it because a cubic spline reproduces quadratics exactly, leaving only rounding error to fit.

TABLE III
THE GRADIENT-ERROR CONVERGENCE RATE AGAINST THE RECONSTRUCTION'S POLYNOMIAL DEGREE, ON A JITTERED MESH. EACH CELL IS A SEPARATE RECONSTRUCTION OF THE SAME NODAL VALUES, AND EVERY DEGREE IS DIFFERENTIATED BY THE SAME RULE – CENTRAL DIFFERENCES ON ITS OWN INTERPOLANT – SO THE ONLY THING VARYING ACROSS A ROW IS THE RECONSTRUCTION. THE RATE RISES WITH THE DEGREE AND THE ERROR AT FIXED MESH FALLS BY FOUR TO FIVE ORDERS OF MAGNITUDE, WHICH MAKES THE PAPER'S RECOMMENDATION ACTIONABLE BEYOND THE TWO RECONSTRUCTIONS IT WAS ORIGINALLY TESTED ON.

degree	exp.	trig.	rational
1	1.05	1.03	1.10
3	3.38	3.53	3.29
5	5.08	6.00	6.34

The order is a property of the reconstruction, and two points are not enough to show it. The comparison above rests on multilinear against cubic, and two reconstructions can be ordered by accident. Table III generalises it: the same nodal values are reconstructed at four polynomial degrees, on three test functions, on a *jittered* mesh, and – the part that makes the comparison mean anything – every degree is differentiated by an identical rule, central differences on its own interpolant, so nothing but the reconstruction varies across a row. The measured gradient convergence rate rises monotonically with the degree, from 1.03–1.10 at degree one to 3.29–3.53 at degree three and 5.08–6.34 at degree five, and at a fixed mesh the error falls by four to five orders of magnitude. The claim therefore holds along the whole sequence rather than between two chosen representatives, which is what makes the recommendation actionable: a practitioner who wants control accuracy does not choose a better solver family, they raise the order of the operator that turns their stored values into a gradient.

One caution belongs with the degree-five row. On the exponential it first measured 2.89 rather than the 5–6 the other functions give, and the reason is not the reconstruction: at that degree the error reaches the floating-point floor around 10^{-10} and plateaus, so the last few mesh sizes measure rounding rather than convergence and a fit across them reports an order that is far too low. The reported figure excludes the plateau. The same trap produced a spurious order of -2.1 in the earlier comparison, where a cubic reconstruction is exact on a quadratic and leaves nothing but rounding to fit.

Refining in time cannot substitute for any of this. Holding the mesh fixed and varying only the number of stored snapshots separates the two errors cleanly. The value error falls by two to four orders of magnitude ($\times 23$, $\times 163$, $\times 428$ at $p = 2$ for $N = 21, 35, 45$; $\times 4{,}559$, $\times 17{,}269$, $\times 4{,}249$ at $p = 3$), while the control error falls by only $\times 3.8$, $\times 10$ and $\times 17$ at $p = 2$ and then stops improving; at $p = 5$ it does not improve at all. The plateau is not arbitrary: at $p = 2$ it scales as h^2 to within measurement error.

The resolution an application needs is therefore fixed by the control target *and* the order of the operator used to differentiate the solution — and the second of those is free at the grid level. That is not an argument that control error is harmless: the envelope theorem makes a control error δu perturb the value only by $O(\|\delta u\|^2)$, while the inventory path is a time integral of the control and inherits its error to *first* order. A solver can therefore be accurate enough for valuation and still execute the wrong inventory.

Measured on the executed path, that is what happens. The argument above is an argument; it can be turned into a measurement by closing the loop. Integrating $dx = (Ax + Bu)\,dt$ from a liquidation start under the control each reconstruction actually produces, and comparing against the exact closed loop, separates the consequences cleanly at $N = 11$ (signalled benchmark): the multilinear extraction gives a maximum position error of 1.6×10^{-2} of the initial position and an objective loss of 1.35% of $|J^\star|$, while the cubic extraction on the same nodes gives 4.2×10^{-3} and 3.9×10^{-4}. The ratio between the two quantities grows with refinement — 10.7, 31.3, 55.9, 68.4 over $N = 11, \ldots, 31$ — and the fitted orders are what the envelope theorem predicts rather than merely consistent with it: the position error converges at 1.7–2.0, the objective loss at 2.1. In other words an improvement that a value benchmark cannot see, a $35\times$ reduction in the objective loss, is accompanied by a $4\times$ improvement in the position actually held, and it is the position that a risk limit or an inventory constraint acts on. One limit is worth stating: the controller is stored as snapshots and blended in time, so its time sampling contributes about 10^{-3} to the position error — comparable to the finest-mesh figures — and the fine-mesh rows of that comparison are therefore time-limited rather than reconstruction-limited.

C. Dimension and parameterisation

The sweep above stops at $p = 5$ because grid methods stop being runnable past that. The analytic reference does not: the Riccati equation is a fixed number of ODEs whatever the state dimension, so accuracy can still be *measured* where no classical baseline can be run, and we extended the range to $p = 20$. Doing so exposed a flaw in the sweep above. With the uniform defaults the value function is *separable*, $V(x, t) = \sum_i v(x_i, t)$, so solving it at $p = 20$ is barely harder than at $p = 1$: the network applies one learned univariate function

and sums. That is not the case the curse-of-dimensionality argument is about.

The signal extension of Section III-A removes the symmetry by construction rather than by assumption — with *perfectly uniform* parameters the off-diagonal block of P reaches 0.205 of the diagonal, against exactly zero here — and the contrast at *matched state dimension*, with the same architecture, budget and two seeds, is stark:

d	separable	with a signal
2	$+0.9994 \pm 0.0002$	$+0.9993 \pm 0.0002$
4	-44.5 ± 41.2	$+0.9988 \pm 0.0001$
6	-646.6 ± 0.3	$+0.9968 \pm 0.0003$

At $d = 4$ and $d = 6$ the separable problem is not merely less accurate but unsolved — a negative R^2 means the learned value function is worse than predicting the mean — while the harder, genuinely multi-dimensional problem is solved to three decimals. Randomising each asset's risk and impact coefficients reproduces the same contrast on the inventory-only problem (-688 against 0.981 at $p = 8$), which is how we first saw it; the signal version is the one to quote, because it needs neither a random draw nor the claim that a particular draw is representative. The failure is a property of the parameterisation, not of the dimension.

On the non-separable problem the PINN does not collapse; it degrades with dimension, giving $R^2 = 0.99999$, 0.9994, 0.9992, 0.977, 0.981, 0.927 and 0.757 at $p = 1$, 2, 3, 5, 8, 12 and 20. A grid baseline remains available and more accurate through $p = 5$ (0.9999 down to 0.9993) but is not runnable at $p \geq 8$, where N^d alone is 2×10^7 points before the CFL-limited time step is counted. Beyond that the mesh-free solver is the only method that runs at all — at accuracy that declines to 0.76 by $p = 20$ and that the user cannot bound, because nothing is left to compare against except the analytic solution we happen to have here.

The practical guidance is therefore narrower than "PINNs fail at high dimension". What fails is a combination of architecture, parameterisation and budget, and the same architecture succeeds at the same dimension once the problem is made generic — so a PINN validated only on a symmetric toy problem has learned nothing about either outcome.

One objection is under-training, and it deserves a direct answer. At $p = 4$ the fixed budget of 6000 epochs and 2048 collocation points yields $R^2 = -44.5$ in 177 s; 20000 epochs at the same batch yields $R^2 = +0.9972$ in 630 s. At $p = 3$ the same increase moves R^2 from 0.9981 in 172 s to 0.9997 in 541 s. Budget is therefore one lever. It is not the only one, and saying so was an over-reach in an earlier draft of this paragraph: Section V-D shows that at the *unchanged* budget a deeper network of the same family also recovers this dimension, so the failure is a statement about the combination in front of it, not about budget alone. What does not change under either route is the cost: the rescued $p = 4$ run took 630 s against 31.7 s for accuracy-matched finite differences — about $20\times$ more expensive, for lower accuracy ($R^2 = 0.9972$ versus 0.99991). Since the neural solver's cost grows only slowly in p while the grid scheme's grows geometrically, the gap must close eventually, but over $p = 1$ to 5 it does not, and at each dimension where both methods succeed the grid scheme is faster by a factor of 5 to 50 — at the coefficients benchmarked here, a scope Section V-E measures.

TABLE IV

ARCHITECTURE SENSITIVITY AT $p = 4$, THE DIMENSION WHERE THE FIXED BUDGET COLLAPSES. EVERY ROW IS A FULL SOLVE SCORED AGAINST THE ANALYTIC REFERENCE; THE FIRST ROW IS THE ACCURACY-MATCHED GRID SCHEME AT THE SAME DIMENSION, FOR SCALE. TWO SEPARATE QUESTIONS ARE ANSWERED HERE. DOES MORE CAPACITY HELP AT A FIXED BUDGET – YES, DECISIVELY: AT 6,000 EPOCHS THE PAPER'S 4×64 SCORES $R^2 = -3.28$ WHILE AN 8×64 MLP, USING $2.3\times$ THE PARAMETERS, SCORES $+0.9953$. AND IS CAPACITY THE ONLY LEVER – NO: RAISING THE BUDGET OF THE ORIGINAL ARCHITECTURE TO 20,000 EPOCHS REACHES $+0.9972$ FOR LESS WALL TIME THAN THE LARGEST NETWORK HERE. BOTH ROUTES SUCCEED, AND BOTH REMAIN SLOWER AND LESS ACCURATE THAN THE GRID SCHEME.

net	ep.	params	time (s)	value R^2	control R^2
grid ($N = 21$)	—	—	20	$+0.99991$	$+0.99603$
2x64 shallow	6k	4,609	61	-337.02583	-1.0963
4x64 (paper)	6k	12,929	85	-3.27683	$+0.7316$
8x64 deep	6k	29,569	128	$+0.99530$	$+0.9969$
4x128 wide	6k	50,433	79	$+0.93647$	$+0.9869$
6x256 both	6k	330,753	100	$+0.99672$	$+0.9977$
2x64 shallow	20k	4,609	208	$+0.98600$	$+0.9874$
4x64 (paper)	20k	12,929	290	$+0.99719$	$+0.9976$
8x64 deep	20k	29,569	450	$+0.99956$	$+0.9993$
4x128 wide	20k	50,433	288	$+0.99974$	$+0.9992$
6x256 both	20k	330,753	351	$+0.99757$	$+0.9976$

D. How much does the architecture choice matter?

Table IV answers the objection this paper would otherwise have to concede, that a single architecture cannot separate a property of the solver family from an artefact of the network. The sweep is run at $p = 4$, the dimension where the fixed budget collapses, because that is where an architecture effect has room to show; a dimension where the solver already succeeds would measure nothing. Each row is a full solve, and the grid scheme is included only for scale.

Three things follow, and the third is the one that matters.

Capacity does move the boundary. At the paper's own 6000-epoch budget the 4×64 MLP scores $R^2 = -3.28$ while an 8×64 MLP scores $+0.9953$ for $2.3\times$ the parameters and $1.5\times$ the wall time. A collapse at a fixed budget is therefore not evidence that the family is capacity-limited at that dimension; it is evidence that this architecture is.

Depth is the lever, not width. At the same budget the 8×64 network (29,569 parameters) reaches $+0.9953$ while the wider 4×128 (50,433 parameters) reaches only $+0.9365$. More parameters bought a worse result, so the useful move is deeper composition of the scaled inputs rather than a bigger final layer.

Both routes work, and neither reaches the grid. Raising the budget of the *original* architecture to 20000 epochs reaches $+0.9972$ in 290 s. The 331k-parameter network is more accurate at $+0.9976$ but costs 351 s to get there, so on

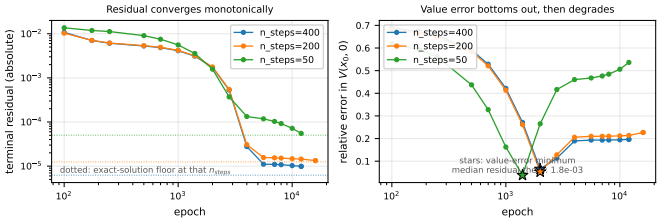


Fig. 3. Deep BSDE: the terminal residual converges, the value does not. Left, the residual falls monotonically at every resolution, approaching the discretisation floor of the scheme it is solving (dotted). Right, the same runs' relative error in $V(x_0, 0)$ bottoms out and then *grows* (stars mark each minimum). Training to a residual plateau therefore overshoots the best value the run passes through, by a factor of 14 to 686 depending on dimension — which is why Table V quotes the value at its minimum and not at the end.

TABLE V

DEEP BSDE, THE STANDARD HIGH-DIMENSIONAL METHOD IN MATHEMATICAL FINANCE, ON THE SAME PROBLEM AND AT THE SAME DIMENSIONS AS TABLE I. TWO COLUMNS ARE DELIBERATELY NOT COMPARABLE WITH THAT TABLE AND ARE LABELLED SO. FIRST, THE ERROR IS MEASURED AT THE SINGLE INITIAL STATE THE SOLVER ANSWERS FOR (x_0, ONE UNIT OF EACH ASSET), WHEREAS THE GRID AND PINN COLUMNS OF TABLE I ARE DOMAIN-WIDE: A POINT-VALUED SOLVER IS NOT BEING CREDITED WITH A FUNCTION-VALUED ONE'S ACCURACY. SECOND, THE VALUE IS QUOTED *at its best point during training*, NOT AT THE END. IN EVERY RUN THE TERMINAL RESIDUAL KEPT FALLING WHILE THE VALUE ERROR BOTTOMED OUT AND THEN DEGRADED, SO THE ENDPOINT IS A FACTOR OF 14 TO 686 WORSE THAN THE NUMBER HERE, AND A READER WHO TRAINS TO A RESIDUAL PLATEAU WILL NOT REPRODUCE THIS COLUMN. TIME IS TRAINING WALL TIME TO THAT STOPPING POINT, AT THE PER-EPOCH RATE MEASURED FOR EACH DIMENSION IN A SINGLE CONTROLLED RUN; THAT RATE IS NEARLY FLAT IN p (0.11–0.15 S PER EPOCH), SO THE COST IS CARRIED BY THE NUMBER OF EPOCHS NEEDED TO CONVERGE, NOT BY THE COST OF AN EPOCH. ONE TRAINING RUN ANSWERS ONE INITIAL STATE; IT DOES NOT RETURN V OVER THE DOMAIN.

p	n_{steps}	train (s)	rel. err at x_0	domain var	resid. at stop
1	200	309	0.053	4.22e-06	1.76e-03
4	50	696	0.003	1.62e-05	7.17e-01
8	50	759	0.003	2.76e-05	7.17e-01

accuracy per second the small network given more steps is the better trade — which is the direction a practitioner should take from this table. At 20000 epochs four of the five architectures exceed $+0.9995$ and the architecture effect is small: what looked like a capacity story at 6000 epochs is largely a budget story, exposed by giving capacity somewhere to go. Across all eleven configurations the best neural result is $+0.99974$ in 288 s, against the accuracy-matched grid at $+0.99991$ in 20 s. The separation this paper reports therefore does not depend on which network is chosen, which is the claim the single-architecture design could not previously support.

The sweep is over MLPs at a fixed optimiser and a fixed collocation scheme. It does not exclude operator architectures, spectral features or curriculum on the sampling, and the honest reading of "no architecture search" is that this bounds the effect of capacity within one family rather than the effect of architecture as such.

The omission that most needs answering is deep BSDE, which is the standard high-dimensional method in mathematical finance and which we therefore ran rather than merely cited. Table V gives it on the same problem at $p = 1$, 4 and 8, and the comparison is deliberately narrow: deep BSDE is point-valued, so its error is measured at the single initial state it answers for, not over the domain as the grid and PINN columns of Table I are. Two things follow.

First, where the grid scheme can run, deep BSDE does not reach its accuracy. At $p = 1$ and $p = 4$ its best value error is 5.3% and 3.3%, against value R^2 of 0.99995 and 0.99992 for accuracy-matched finite differences, and its terminal residual at that point is still $71\times$ and $2.6 \times 10^4\times$ the discretisation floor of the scheme it is solving. So the conclusion of Table I is not an artefact of having chosen the PINN as the neural representative: the other standard neural family, measured rather than assumed, loses to the classical scheme on the same axis in the regime where the classical scheme is available.

Second, and in the other direction, deep BSDE is the strongest of the learned solvers where the grid is not runnable. At $p = 8$ it reaches 0.35% relative error on $V(x_0, 0)$, while the PINN reaches $R^2 = 0.977$ and the grid scheme is 2×10^7 points before its CFL-limited time step is counted. The cost is not the per-epoch rate — that is nearly flat in p, 0.11 to 0.15 s — but the number of epochs needed to reach the stopping point, which rises from 2000 at $p = 4$ to 6000 at $p = 8$.

One caution belongs with any deep-BSDE number, and Figure 3 is the reason. In every run the residual kept falling while the value error degraded, reaching 1268% by the end of the $p = 4$ run. The stopping point is consequently part of the reported configuration. It does not transfer between dimensions: the absolute residual threshold that is optimal at $p = 1$ is three orders of magnitude too late at $p = 4$. A scale-free version still moves by a factor of 31 across the dimensions we measured, so the stopping point has to be found by monitoring the value rather than by a rule.

E. Market-calibrated scope

Everything above uses coefficients that were chosen, not measured: $\eta = \lambda\sigma^2 = 0.01$ and $Q_T = 0.1$. A benchmark is entitled to do that, but the entitlement expires when its conclusions are read as statements about trading, so this subsection asks what happens when the coefficients come from the data instead.

Two of the three are measurable and one is not. The diffusion scale is the relative volatility, which we take from the minute bars. The impact coefficient is not observable, and we identify it with half the Roll effective spread [34] – a round trip pays the spread once, so a unit trade pays about half of it – using the estimator already computed on the causal data and cross-checked against Corwin–Schultz [35]. Risk aversion is a preference. Nothing in a price series determines it, so any single calibrated value would be a choice dressed as a measurement.

That last point sets the design rather than excusing it. On this problem the pinned quantity is $k = \sqrt{Q/\eta} = \sigma\sqrt{\lambda}/\sqrt{\eta}$, which fixes how fast the position is liquidated. Measured liquidity with a defensible $\lambda \in [10^{-6}, 10^{-2}]$ puts k between

TABLE VI

THE EXPLICIT GRID SCHEME'S TIME-STEP REQUIREMENT UNDER MEASURED LIQUIDITY. η IS HALF THE ROLL EFFECTIVE SPREAD [34] MEASURED ON THE CAUSAL MINUTE DATA, WITH CORWIN–SCHULTZ [35] AS A CROSS-CHECK. THE OPTIMAL CONTROL IS $u^\star = -R^{-1}B^\top Px$ WITH $R = \eta I$, AND THE EXPLICIT TIME STEP IS CFL-LIMITED BY $dt \le 0.25\,dx/\|u^\star\|_\infty$, SO A SMALLER η BUYS A LARGER CONTROL AND A PROPORTIONALLY SMALLER STEP. EVERYTHING ELSE IS HELD AT THE PAPER'S SYNTHETIC VALUES, SO THE COLUMN IS A STATEMENT ABOUT THE INSTRUMENT'S LIQUIDITY ALONE. THE ORDERING IS THE POINT: THE MOST LIQUID INSTRUMENT PAYS THE LARGEST PENALTY.

instrument	half-Roll spread η	$\max \lvert u^\star \rvert$ on grid	time steps	vs. synthetic calibration
synthetic	1.00e-02	12.5	800	1×
SPY	1.28e-04	975	62,427	78×
AMZN	1.46e-04	858	54,889	69×
MSFT	1.58e-04	792	50,713	63×
QQQ	1.60e-04	781	50,016	63×
GOOGL	1.97e-04	634	40,547	51×
OXY	2.24e-04	557	35,667	45×
NVDA	2.69e-04	466	29,795	37×
AAPL	2.73e-04	458	29,342	37×
TSLA	4.41e-04	283	18,124	23×

0.001 and 0.15; our synthetic coefficients sit at $k = 1$. The benchmark therefore liquidates roughly an order of magnitude faster than real liquidity and realistic risk aversion imply. We sweep λ across that range rather than picking from it, so the results below are a statement about robustness and not about a chosen calibration.

Accuracy is unaffected. Across thirteen configurations – SPY, AAPL, NVDA and TSLA, each at three values of λ, at $p = 1$ – the grid scheme scores value R^2 between $+0.99996$ and $+0.99997$ against $+0.99999$ at the synthetic coefficients, with control R^2 unchanged at $+0.9990$. The control-to-value error ratio, the quantity Section V-B is built on, lands between 7.9 and 16.4 across every configuration, against 16.4 synthetic. Real coefficients move neither the accuracy of the classical scheme nor the size of the control/value gap.

Cost does not survive, and this is the finding. The optimal control is $u^\star = -R^{-1}B^\top Px$ with $R = \eta I$, so a smaller impact coefficient buys a proportionally larger control; the explicit scheme's step is CFL-limited by $dt \le 0.25\,dx/\|u^\star\|_\infty$; and the step count therefore scales as $1/\eta$. Table VI evaluates that at each instrument's own measured spread, holding everything else at the paper's values. The tightest instrument in the collection, SPY, needs $78\times$ the synthetic run's time steps; the widest, TSLA, needs $23\times$. The ordering is the opposite of the intuitive one: liquidity is what makes the explicit scheme expensive here, not illiquidity.

So the paper's speed claim has a narrower scope than its accuracy claim, and we state it that way. Where a grid scheme can run it is faster than the mesh-free solver *at the coefficients benchmarked here*; transferring that to a specific instrument requires the step count to be redone at that instrument's spread. Two qualifications keep this from over-reading. It constrains the *explicit* scheme, which is the one our cost tables use; our implicit variant (Section VI) removes the stability restriction at four times the explicit step, but it was measured at the synthetic η only, so nothing here says an implicit scheme is slow at real spreads. And a step count is work, not wall-clock: the grid's per-step cost is the point count, so the ratio of step counts is the ratio of work for a fixed mesh.

One methodological consequence is worth recording because it cost us a wrong result first. The accuracy function samples query times with `linspace`, which includes $t = T$, and the reference control diverges there – the last instant of a liquidation problem wants to trade infinitely fast. At the synthetic η the terminal sample is mild, with mean $|u^\star| = 5.1$ against 0.6 to 1.7 in the interior, and excluding it moves control R^2 by less than 10^{-4}. At SPY's η, smaller by $78\times$, the terminal sample reaches 395 and a solve that is accurate in the interior is scored at value $R^2 = -997$. The estimator is therefore not the difficulty; the sampling is, and any evaluation at real spreads has to drop the terminal sample.

VI. DISCUSSION AND LIMITATIONS

A. When to use which

Table VII states the recommendation the comparison supports. Every row is a regime a practitioner can recognise before committing to a method, and every recommendation points at a measurement in this paper rather than at a preference. The neural solver is preferred in only two rows, and in both the reason is that the classical scheme cannot be run or should not be re-run — not that it was beaten on accuracy; where the two can be compared, the classical scheme wins on both axes.

B. One problem or a family

A natural objection is that the comparison should use inference time rather than training time, since a trained network evaluates in microseconds. We measured both, and the objection does not hold here: both methods have an expensive offline solve followed by cheap online queries. Amortised over one trading day of minute decisions the grid scheme's interpolation costs $0.66\,\mu$s at $p = 1$ and $34.3\,\mu$s at $p = 5$, against 3.0–4.9 μs for the PINN on GPU, so the neural solver's inference is *not* uniformly faster. Combining the terms as $T(N) = T_{\text{offline}} + N\,t_{\text{eval}}$, the PINN never breaks even at $p = 1$, and at $p = 5$ it is finally cheaper on *both* terms (226 s against an 865 s solve) only where it is not accurate ($R^2 = -616$): the binding constraint has moved from cost to accuracy.

The regime changes for a *family* of problems. Where the solver must be re-solved for every member, the break-even count with 10^3 training solves is $4{,}365$ instances at $p = 2$ and $1{,}000$ at $p = 5$ — essentially the size of the training set, and *falling* as dimension rises — so an *operator* amortises after $1{,}000$ to $4{,}365$ parameter settings, which is exactly the regime a calibration study occupies. A fixed-parameter PINN does not, and most operator architectures would need labelled solutions from the solver they aim to replace, though the physics-informed family [37] trains on the residual and needs none.

TABLE VII

WHICH SOLVER TO USE. EACH ROW IS A REGIME A PRACTITIONER CAN RECOGNISE BEFORE SOLVING ANYTHING, AND THE EVIDENCE IS A MEASUREMENT REPORTED IN THIS PAPER RATHER THAN A PREFERENCE. THE RECOMMENDATION IS DELIBERATELY CONDITIONAL: THE TWO ROWS THAT PREFER THE NEURAL SOLVER ARE CASES WHERE THE CLASSICAL SCHEME *cannot* BE RUN OR *should not* BE RE-RUN, NOT CASES WHERE IT IS BEATEN ON ACCURACY.

Regime	Use	Deciding measurement
Single instance, $p \leq 5$	**grid scheme**	faster *and* more accurate everywhere it runs: factor 5–50 in time at $p = 4$–5, two to three orders at low p (Table I)
Control accuracy matters, not just value accuracy	**grid scheme**, and raise the reconstruction order	a cubic reconstruction on the same nodes cuts the control error 4–5 orders of magnitude and the control/value ratio from 185 to 2.8 (Table II)
$p \geq 8$	**neural**, the only method that runs	the grid needs N^p points; at $N = 21$, $p = 8$ is 2×10^7 before the CFL-limited time step is counted (Section V-C)
A family of $\gtrsim 10^3$ problems	**learned operator**	the classical scheme must be re-solved per instance; an operator amortises after 1,000 to 4,365 parameter settings
Accuracy must be known, not assumed	**grid scheme**	the neural solver's accuracy decays from $R^2 = 0.98$ at $p = 8$ to 0.76 at $p = 20$ while its control-sign agreement stays near 0.7: no internal signal of the decay (Section V-C)
Inputs will leave the training domain	**scale them explicitly**, whichever solver	an unscaled network's price sensitivity decays monotonically with price level, rank correlation -0.90 to -1.00, so any threshold on it yields an artefact (Section VI-C)

C. Out-of-domain evaluation

A surrogate trained on a normalised state domain and evaluated in physical units fails quietly. A 4×64 tanh MLP trained on the HJB residual with terminal condition $V(S,T) = -\frac{1}{2}(S-25)^2$ on prices sampled from $[20, 30]$ only, then evaluated on the realised price levels of the nine instruments ($42 to $536), shows a learned price sensitivity $|\partial V/\partial S|$ that falls monotonically with price — from 3.36 at $42 to 0.062 at $536, a factor of about 54 — with rank correlations of -0.90, -0.98 and -1.00 across three seeds. The network saturates in proportion to how far its input lies beyond the training domain, so any threshold applied to this gradient inherits the ordering and produces a cross-sectional "finding" that admits a plausible microstructure story while being entirely an artefact of scale. Mapping the input back into the training domain removes the dependence ($+0.12$, -0.22, $+0.23$). What we add to the known out-of-domain failure of learned operators [38] is its *shape* in a control setting — saturation rather than divergence, so the damage survives every sanity check that looks for obviously broken output — and the procedural recommendation that follows: report per-channel input coverage for every evaluation set.

Limitations. (i) The cost half of the argument stops at $p \approx 5$, where the grid scheme stops being runnable at the resolution the accuracy target needs. That limit deserves one qualification, because the obvious objection to it is the wrong one. The restriction is not the CFL condition as such: we implemented the diffusion implicitly, by one-dimensional splitting, and it is stable and accurate at four times the explicit step (value R^2 still 0.999). But splitting is only first-order in time, so holding the temporal error below the spatial error still requires $dt \sim dx^2$ — the same rate the CFL condition imposes — and what implicitness buys is a constant factor of about four rather than a change in the scaling. The limitation is therefore the cost of controlling the temporal error, not stability; moving it needs a second-order time integrator, or sparse grids, which attack the N^d factor instead. (ii) The neural results use one architecture (4×64 tanh), one optimiser and one schedule. Section V-D measures what that choice is worth *within* the family — capacity and budget each move the boundary at $p = 4$ — but the sweep is still over MLPs at a fixed optimiser and fixed collocation, so it bounds the effect of size rather than of architecture as such. Techniques for high-dimensional PDEs [21], [22] could plausibly shift where the boundary sits, and so could architectural choices deliberately left out of scope here – wider residual blocks [39], or input feature maps that change the network's spectral bias and therefore which scales it can represent [40] – and we release the benchmark so that such claims can be tested against a closed-form reference. (iii) The separability effect is established as a phenomenon, not explained: we do not claim that symmetry *causes* the failure. (iv) Timings come from a single GPU and CPU, and the comparison is stated as GPU-for-neural versus CPU-for-grid, which favours the neural method. (v) The control-accuracy result assumes a solver whose control is a finite difference of an interpolated value function; a scheme that solves for the control directly could narrow the gap, and we have not tested it. (vi) The model coefficients are synthetic rather than calibrated to an instrument, so no result here is a statement about how a particular asset trades. Section V-E measures what calibration does to the conclusions, and the answer is not uniform: the *accuracy* comparison is unchanged, while the *cost* comparison is not, because a market-calibrated impact coefficient multiplies the explicit scheme's time-step requirement by 23–78$\times$ depending on the instrument. The speed claim therefore carries the scope stated in Section V-E and not a general one. What the paper supplies no evidence about, under any calibration, is realised trading performance, and we offer none.

VII. Conclusion

We compared neural and classical solvers for the HJB equation of multi-asset optimal execution against a closed-form Riccati reference. The comparison produces a boundary rather than a winner, because the two families are not ranked along one axis but along three. Wherever a grid scheme can run, up to about five state dimensions, it is both faster and more accurate, by 5–50$\times$ in time at $p = 4$–5; beyond that it is not runnable, and the mesh-free solver is the only method that runs, at accuracy decaying from $R^2 = 0.98$ at $p = 8$ to 0.76 at $p = 20$ with no internal signal that it is decaying.

Measuring the *control*, which is what an HJB is for, says something more useful: value accuracy does not bound control accuracy, because what limits the control is the order of the operator that turns nodal values into a gradient. A multilinear reconstruction is second order, its exact gradient is also second order, and a cubic reconstruction on the same nodes is fourth order and four to five orders of magnitude more accurate, cutting the control/value error ratio from 184 to 2.8. That improvement is free at the grid level, which is why the actionable recommendation is the second row of Table VII: if control accuracy matters, change the reconstruction before changing the solver.

Two objections changed the picture without reversing it. A larger training budget recovers accuracy a smaller one loses, and we report the rescued run rather than the flattering comparison. And the field's preference for inference time is justified, because a classical solver must be re-solved for every instance — but the two framings answer different questions, one problem or a family of them, and have opposite answers. Both generalise: validate on the problem you actually have, with a reference you can trust.

Reproducibility

Code, the temporal data splits, the preprocessing manifest and every raw result file behind the tables are released at https://github.com/ridleyton/HJB-Solver-Comparison. The reference is checked against three independent closed forms, and the manuscript is built and page-checked by `src/build_paper.py`.

References

[1] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, 2008.

[2] O. Guéant, C.-A. Lehalle, and J. Fernandez-Tapia, "Dealing with the inventory risk: a solution to the market making problem," *Mathematics and Financial Economics*, vol. 7, no. 4, pp. 477–507, 2013.

[3] O. Guéant, "Closed-form approximations in multi-asset market making," *Applied Mathematical Finance*, 2016. volume and pages to confirm.

[4] Á. Cartea and S. Jaimungal, "Risk metrics and fine tuning of high-frequency trading strategies," *Mathematical Finance*, vol. 25, no. 3, pp. 576–611, 2015.

[5] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *Journal of Risk*, vol. 3, no. 2, pp. 5–39, 2000.

[6] Á. Cartea, S. Jaimungal, and J. Penalva, *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.

[7] P. A. Forsyth and G. Labahn, "ϵ-monotone fourier methods for optimal stochastic control in finance," *arXiv preprint arXiv:1710.08450*, 2017.

[8] R. A. Howard, *Dynamic Programming and Markov Processes*. MIT Press, 1960.

[9] H. J. Kushner, "Numerical methods for stochastic control problems in continuous time," *SIAM Journal on Control and Optimization*, vol. 28, no. 5, pp. 999–1048, 1990.

[10] D. P. Bertsekas, *Dynamic Programming and Optimal Control, Vol. I*. Athena Scientific, 4th ed., 2012.

[11] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[12] J. Sirignano and K. Spiliopoulos, "Dgm: A deep learning algorithm for solving partial differential equations," *Journal of Computational Physics*, vol. 375, pp. 1339–1364, 2018.

[13] B. Yu and W. E, "The deep ritz method: A deep learning-based numerical algorithm for solving variational problems," *Communications in Mathematics and Statistics*, vol. 6, no. 1, pp. 1–12, 2018.

[14] W. E, J. Han, and A. Jentzen, "Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations," *Communications in Mathematics and Statistics*, vol. 5, no. 4, pp. 349–380, 2017.

[15] J. Han, A. Jentzen, and W. E, "Solving high-dimensional partial differential equations using deep learning," *Proceedings of the National Academy of Sciences*, vol. 115, no. 34, pp. 8505–8510, 2018.

[16] J. Han and W. E, "Deep learning approximation for stochastic control problems," *arXiv preprint arXiv:1611.07422*, 2016.

[17] C. Beck, W. E, and A. Jentzen, "Deep splitting method for parabolic pdes," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, pp. A3135–A3154, 2021.

[18] M. Fujii, A. Takahashi, and M. Takahashi, "A machine learning approach to Malliavin calculus with applications to stochastic optimal control," *Asia-Pacific Financial Markets*, vol. 26, no. 3, pp. 319–364, 2019.

[19] H. Pham, "Stochastic control problems with BSDEs and machine learning," *SIAM Journal on Financial Mathematics*, 2021. volume and pages to confirm.

[20] M. Hutzenthaler, A. Jentzen, T. Kruse, and T. A. Nguyen, "Overcoming the curse of dimensionality in the numerical approximation of high-dimensional semilinear elliptic partial differential equations," *arXiv preprint arXiv:2003.00596*, 2020.

[21] X. Meng, Z. Li, D. Zhang, and G. E. Karniadakis, "Ppinn: Parareal physics-informed neural network for time-dependent pdes," *Computer Methods in Applied Mechanics and Engineering*, vol. 370, p. 113250, 2020.

[22] B. Moseley, A. Markham, and T. Nissen-Meyer, "Finite basis physics-informed neural networks (fbpinns): A scalable domain decomposition approach for solving differential equations," in *Advances in Neural Information Processing Systems*, 2021.

[23] R. Carmona and M. Laurière, "A machine learning framework for solving high-dimensional mean field game and mean field control problems," *Proceedings of the National Academy of Sciences*, vol. 117, no. 17, pp. 9183–9193, 2020.

[24] H. Bühler, L. Gonon, J. Teichmann, and B. Wood, "Deep hedging," *Quantitative Finance*, vol. 19, no. 8, pp. 1271–1291, 2019.

[25] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *International Conference on Learning Representations (ICLR)*, 2021.

[26] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Learning maps between function spaces with applications to pdes," *Journal of Machine Learning Research*, vol. 24, no. 89, pp. 1–97, 2023.

[27] B. Raonić, R. Molinaro, T. De Ryck, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, and E. de Bézenac, "Convolutional neural operators for robust and accurate learning of pdes," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[28] H. Wu, H. Luo, H. Wang, J. Wang, and M. Long, "Transolver: A fast transformer solver for pdes on general geometries," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235 of *PMLR*, 2024.

[29] X. Han, F. Hou, and H. Qin, "Ugrid: An efficient-and-rigorous neural multigrid solver for linear pdes," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, vol. 235 of *PMLR*, 2024.

[30] P. Wei, M. Liu, J. Cen, Z. Zhou, L. Chen, and Q. Zou, "Pdenneval: A comprehensive evaluation of neural network methods for solving pdes," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[31] F. A. Longstaff and E. S. Schwartz, "Valuing American options by simulation: a simple least-squares approach," *The Review of Financial Studies*, vol. 14, no. 1, pp. 113–147, 2001.
[32] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. Springer, 2003.
[33] T. Xiao, R. Yang, Y. Cheng, and J. Suo, "Shop: A deep learning framework for solving high-order partial differential equations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 16032–16039, 2024.
[34] R. Roll, "A simple implicit measure of the effective bid-ask spread in an efficient market," *The Journal of Finance*, vol. 39, no. 4, pp. 1127–1139, 1984.
[35] S. A. Corwin and P. Schultz, "A simple way to estimate bid-ask spreads from daily high and low prices," *The Journal of Finance*, vol. 67, no. 2, pp. 719–760, 2012.
[36] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.
[37] S. Wang, H. Wang, and P. Perdikaris, "Physics-informed neural operator for learning partial differential equations," *ACM/IMS Journal of Data Science*, 2021. arXiv:2111.03794.
[38] S. C. Mouli, D. C. Maddix, S. Alizadeh, G. Gupta, A. Stuart, M. W. Mahoney, and Y. Wang, "Using uncertainty quantification to characterize and improve out-of-domain learning for pdes," *arXiv preprint arXiv:2403.10642*, 2024.
[39] S. Zagoruyko and N. Komodakis, "Wide residual networks," in *British Machine Vision Conference (BMVC)*, 2016.
[40] S. Wang, H. Wang, and P. Perdikaris, "On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 384, p. 113938, 2021.